

E-Commerce Price Optimization Engine

Project Documentation and Team Context Summary

GitHub Repo: <https://github.com/Aditya-amrahs/E-commerce-Price-opt>

Team Ownership

US-01 to US-05	Aditya Sharma
US-06 to US-09	Anisha Sinha
US-10 to US-14	Urvi Jain

Problem Statement

Retail e-commerce platforms often price products using static rules or manual judgment. That leads to missed revenue opportunities, weak demand response, and inconsistent business decisions. This project aims to create a data-driven price optimization engine that uses historical order behavior to support smarter pricing decisions.

Objective

The objective is to build a clean and reusable ML foundation that ingests e-commerce data, prepares pricing features, trains baseline and advanced models, and later supports recommendation APIs, dashboards, dynamic pricing logic, and cloud deployment.

Dataset Used

Dataset: **Brazilian E-Commerce Public Dataset by Olist (Kaggle)**. It contains 9 CSV files covering orders, order items, products, customers, sellers, reviews, payments, geolocation, and category translation.

Project Structure

Root: shared files such as .env.example, .gitignore, README.md, requirements.txt
data/: raw and processed datasets
artifacts/: feature artifacts, model artifacts, metrics, and reports
ml/: completed US-01 to US-05 data and model pipeline
backend/: reserved for API, pricing engine, and service logic
frontend/: reserved for dashboard and UI work

Brief Summary of User Stories

US	Title	Brief Workaround / Approach
US-01	Data Collection	Load all 9 Olist CSVs into raw SQL tables using SQLAlchemy.
US-02	Data Preprocessing	Clean records, handle missing values, invalid prices, IQR checks, and modeling filters.
US-03	Feature Engineering	Create pricing, demand, time, seller, customer, lag, rolling, and trend features.
US-04	Linear Regression Model	Train clipped Ridge baseline with chronological split and saved residuals.
US-05	XGBoost Model	Train residual XGBoost model and create hybrid prediction bundle.
US-06	Demand Elasticity	Use processed feature-ready CSV to estimate price-demand sensitivity by product/category/month.
US-07	Pricing API	Build FastAPI inference layer using hybrid bundle and reusable prediction service.
US-08	Business Dashboard	Use backend/API outputs and saved metrics to build pricing insight dashboards.
US-09	Dynamic Pricing	Apply rule-based recommendation logic over model predictions and elasticity outputs.
US-10	Competitor Analysis	Add separate competitor comparison pipeline and insight generation.
US-11	Data Storage	Extend schema with migration-ready tables for recommendations, logs, and extra analytics.

US-12	Model Optimization	Tune model and feature subsets while preserving chronological evaluation.
US-13	UI Enhancements	Improve dashboard usability, filtering, and explanation components.
US-14	Azure Deployment	Deploy database, API, and frontend using Azure-ready configuration.

Detailed Overview of US-01 to US-05

- **US-01 Data Collection:** implemented CSV ingestion and raw table loading using ``ml/ingestion/csv_loader.py``, ``ml/ingestion/db_writer.py``, and ``ml/database/engine.py``.
- **US-02 Data Preprocessing:** implemented record cleaning, type handling, IQR checks, delivered-order filtering, and invalid record reporting using ``ml/preprocessing/cleaner.py`` and ``ml/preprocessing/validators.py``.
- **US-03 Feature Engineering:** implemented pricing, freight, demand, lag, rolling, seller, customer, and time features using ``ml/features/engineer.py``; feature store saved via ``ml/features/feature_store.py``.
- **US-04 Linear Regression Model:** implemented chronological split, preprocessing pipeline, clipped Ridge baseline, residual creation, and metric storage using ``ml/preprocessing/pipeline.py``, ``ml/utils/splitter.py``, ``ml/utils/metrics.py``, and ``ml/models/linear_model.py``.
- **US-05 XGBoost Model:** implemented residual learning, hybrid bundle creation, model comparison, and model registry logging using ``ml/models/rgb_model.py``, ``ml/models/hybrid.py``, ``ml/database/model_registry.py``, and ``ml/run_us01_us05.py``.

Real-World Use Case

In a real business setting, this engine can help online retailers estimate fair pricing, compare baseline vs advanced model performance, monitor demand response, and later serve pricing recommendations through APIs and dashboards. This is useful for category managers, pricing analysts, sellers, and business intelligence teams.

Problems Encountered (US-01 to US-05)

- Dataset integration complexity: 9 CSV files had to be cleaned and joined consistently without schema mismatch.
- Data quality issues: missing values, invalid freight or price records, duplicate rows, and noisy product fields required structured preprocessing.
- Leakage risk: some raw features looked strong but actually leaked future or target-adjacent information, so the modeling path had to be tightened.
- Baseline instability: early baseline runs produced unrealistic numbers until the feature set, categorical handling, and clipped log-target baseline were corrected.
- Hybrid overfitting risk: the XGBoost residual model needed stronger regularization, validation-aware training, and bounded prediction logic to stay reliable.

Future Scope

- Demand elasticity modeling for price sensitivity.
- FastAPI-based pricing recommendation service.
- Interactive business dashboard and UI enhancements.
- Dynamic pricing engine with policy and guardrails.
- Competitor comparison logic and extra storage tables.
- Model optimization, hyperparameter tuning, and Azure deployment.

Summary

The project currently delivers a complete foundation for US-01 to US-05 of the E-Commerce Price Optimization Engine. It includes ingestion, preprocessing, feature engineering, baseline regression, and a stronger XGBoost hybrid model. The remaining work should now build on this base through backend API, dashboard, pricing rules, storage extension, optimization, and deployment without duplicating the ML pipeline.

Detailed Overview of US-06 to US-09

- **US-06 Demand Elasticity:** implemented basic price-demand sensitivity logic using processed dataset and created elasticity calculation function in [ml/models/elasticity.py](#).
- **US-07 Pricing API:** implemented FastAPI-based inference service, integrated hybrid model bundle, created [/price](#) endpoint, and enabled dynamic prediction using [backend/main.py](#).
- **US-08 Business Dashboard:** implemented basic visualization module using matplotlib to generate pricing trend charts and used FastAPI [/docs](#) interface for interactive testing and output visualization.
- **US-09 Dynamic Pricing:** implemented rule-based pricing adjustment logic based on demand signals using [backend/pricing.py](#) and integrated it with model predictions in API.

Real-World Use Case

The extended system further supports real-time pricing decisions by exposing model predictions through APIs and applying dynamic pricing strategies based on demand signals. This allows businesses to

automatically adjust prices, analyze price-demand relationships, and visualize pricing trends, making the solution more practical for operational use by pricing teams and business analysts.

Problems Encountered (US-06 to US-09)

- API integration issues: connecting trained hybrid model with FastAPI required correct input formatting and handling missing feature columns.
- Model dependency errors: environment setup issues such as missing libraries (xgboost, sklearn, matplotlib) and version mismatches caused runtime failures.
- Data consistency challenges: identifying correct feature columns from processed dataset for visualization and API input required validation.
- Backend-ML coordination: understanding how to use hybrid bundle instead of direct model prediction required debugging and alignment with ML pipeline.
- Visualization limitations: matplotlib initially failed due to GUI backend issues in FastAPI, which required switching to non-interactive backend.

Future Scope

- Integrate real user input instead of dataset-based simulation.
- Enhance elasticity modeling using statistical or ML-based approaches.
- Develop full-fledged interactive dashboard using React or Streamlit.
- Improve dynamic pricing logic with advanced business rules and constraints.
- Enable deployment of API and dashboard on cloud platforms.

Summary

Additionally, US-06 to US-09 extend this foundation by introducing demand elasticity, pricing API, dynamic pricing logic, and basic visualization, making the system more practical and business-oriented.

Detailed Overview of US-10 to US-14

- **US-10 Competitor Analysis:** Implemented competitor pricing comparison logic using processed dataset. Average product-level competitor prices were calculated and merged with seller pricing data. Developed comparison logic to classify pricing as higher, lower, or equal, along with percentage difference and recommendation generation. Implemented using backend/services/analysis_service.py.
- **US-11 Data Storage:** Designed basic data storage structure for pricing and product-related data. Utilized CSV-based storage and existing dataset pipeline, with scope for extension to SQL-based storage. Schema

considerations include product, seller, pricing, and demand-related attributes. Initial groundwork aligns with ml/database/ modules.

- **US-12 Model Optimization:** Improved prediction performance by evaluating model outputs and tuning prediction strategy. Implemented error analysis using metrics such as MSE, RMSE, R^2 , and percentage error. Applied safe prediction strategies, fallback logic, and feature-based simplifications to stabilize predictions. Optimization logic integrated in backend/services/model_service.py.
- **US-13 UI Enhancements:** Developed an interactive frontend dashboard using React and Vite. Implemented multiple pages including Dashboard, Analysis, Predict, Optimize, Summary, and Adjust Pricing. Integrated API endpoints for real-time data fetching and visualization. Improved user experience through navigation, charts (Chart.js), and structured UI components located in frontend/src/.
- **US-14 Azure Deployment:** Prepared the application for cloud deployment by structuring backend APIs and frontend integration. Ensured modular design for deployment readiness. Deployment can be extended using Azure App Services or similar platforms, with backend (FastAPI) and frontend (React) hosted separately.

Real-World Use Case

These components enhance the system into a practical pricing intelligence platform. Businesses can compare their prices against competitors, optimize pricing strategies, visualize insights through dashboards, and expose predictions via APIs. The system supports pricing analysts, e-commerce managers, and decision-makers in making data-driven pricing decisions.

Problems Encountered (US-10 to US-14)

- **Competitor data approximation:** No direct competitor dataset available, so proxy logic using average product pricing had to be designed.
- **Storage limitations:** Lack of structured database integration limited scalability and required reliance on CSV-based storage.
- **Model tuning constraints:** Since the base model was pre-trained, optimization had to be done externally without retraining, requiring creative fallback strategies.
- **Frontend-backend integration:** Ensuring API compatibility with frontend components required debugging data formats and endpoint responses.
- **Visualization challenges:** Synchronizing backend-generated charts and frontend chart libraries required handling static files and API triggers.

Deployment readiness: Environment dependencies and configuration differences posed challenges for smooth cloud deployment

Future Scope

- Implement real competitor data integration via APIs or external datasets
- Replace CSV storage with full SQL/NoSQL database system
- Perform advanced hyperparameter tuning and retraining pipelines
- Enhance UI with real-time updates, filters, and better visualizations
- Deploy complete system on Azure with CI/CD pipelines
- Introduce authentication and user-specific dashboards

Summary

US-10 to US-14 extend the system into a full-stack, business-ready solution. Competitor analysis introduces market comparison, model optimization improves prediction reliability, UI enhancements provide usability, and deployment readiness ensures scalability. Together, these components transform the project from a machine learning pipeline into a complete e-commerce pricing system.